F1 Data Analysis and Regression

Junyu Teoh
Advised by Dr.Frank Lynch
August 12, 2021

Eastern Washington University
United States
8/12/2021

1 Introduction

Formula-1 (F1) is often regarded as the highest level of motorsports. 10 teams with 2 drivers each go from track to track during a racing season to fight for the drivers and constructors championship title. Compared to other motorsports such as NASCAR or World Rally Championship, F1 cars are both faster and more fragile. NASCAR features cars with larger chassis and more body panels that allow for rougher racing and overtaking. This is why we often see drivers bump into each other in NASCAR. F1 on the other hand, have comparatively small chassis and also feature less robust body panels. If a F1 driver were to bump into another, its more than likely that both drivers will have their cars damaged and negatively impact their race. World Rally Championship has events on all sorts of terrain and will continue racing regardless of the weather. F1 will only race on sanctioned and prepared race tracks. It is also normal for F1 races to be cancelled due to bad weather.

The goal of our research is to first explore our dataset to discover possible hidden relationship between winning and losing, but also to create and evaluate a model to predict win status. To accomplish this, we analyze race data from the year 2014 to 2019. We do not look at data older than 2014 as those races featured a different engine specification than what is permitted today, and nothing after 2019 as 2020 was a short season due to Covid-19 and the 2021 season is not over at the time of this research.

2 Data Collection

Data was retrieved from *ergast.com* using simple terminal commands and saved into various *.csv* files.

3 Description of Tables

1. circuits.csv
   (a) circuitId - Primary key
   (b) circuitRef - Reference name of track
   (c) name - Full name of track
   (d) location - City of track
   (e) country - Country of track

(f) lat/lng/alt - Geographical location of track
(g) url - Link to wikipedia entry of track

2. constructor_results.csv
   (a) constructorResultsId - Primary key
   (b) raceId - Foreign key of races
   (c) constructorId - Foreign key of constructors
   (d) points - Points earned by particular constructor in particular race
3. constructor_standings.csv
   (a) constructorStandingsId - Primary key
   (b) raceId - Foreign key of race
   (c) constructorId - Foreign key of constructors
   (d) points - Points accumulated so far by a particular constructor
   (e) position - Position of particular constructor in the season
   (f) wins - Number of wins by particular constructor so far
4. constructors.csv
   (a) constructorsId - Primary key
   (b) constructorRef - Reference name of the constructor
   (c) name - Full name of the constructor
   (d) nationality - Nationality of the constructor
   (e) url - Link to wikipedia entry of track
5. driver_standings.csv
   (a) driverStandingId - Primary key
   (b) raceId - Foreign key of races
   (c) driverId - Foreign key of drivers
   (d) position - Position of driver in particular championship
   (e) wins - Number of wins so far
6. drivers.csv
   (a) driverId - Primary key

(b) driverRed - Reference name of driver
(c) number - Number chosen by driver
(d) code - Acronym of driver's name
(e) forename - Firstname of driver
(f) surname - Lastname of driver
(g) dob - Date of birth of driver
(h) nationality - Nationality of driver
(i) url - Link to wikipedia entry of driver

7. lap_times.csv

   (a) raceId - Foreign key of races
   (b) driverId - Foreign key of driver
   (c) lap - Number of lap for the entry
   (d) time - Laptime for the driver
   (e) milliseconds - Time in milliseconds

8. qualifying.csv

   (a) qualifyId - Primary key
   (b) raceId - Foreign key of races
   (c) driverId - Foreign key of drivers
   (d) constructorId - Foreign key of constructors
   (e) number - Driver chosen number
   (f) position - Position of driver in particular qualifying
   (g) q1 - Qualifying session 1 fastest lap time
   (h) q2 - Qualifying session 2 fastest lap time
   (i) q3 - Qualifying session 3 fastest lap time

9. races.csv

   (a) raceId - Primary key
   (b) year - Year of race
   (c) round - Round race within a championship
   (d) circuidId - Foreign key of circuit
   (e) name - Name of circuit
   (f) date - Date of race
   (g) time - Time of race
   (h) url - Link to wikipedia entry of race

4 Data analysis

4.1 Qualifying position

Before every race, each driver goes through qualifying. Qualifying is split into 3 18-minute sessions, referred to as Q1, Q2 and Q3. In Q1, 20 drivers drive on the track trying to set the fastest time. The slowest 5 cars are eliminated at the end of the session, and their positions will be the starting position during the race.

Generally, it is desired to start on pole as this means the driver do not have to worry about other drivers and possibly having other drivers crash into them, but more importantly is that the car will have no aerodynamic disturbances from trailing another car[1]. Further, it allows the driver on pole to choose the most ideal racing line instead of being forced into a less ideal line due to other cars.

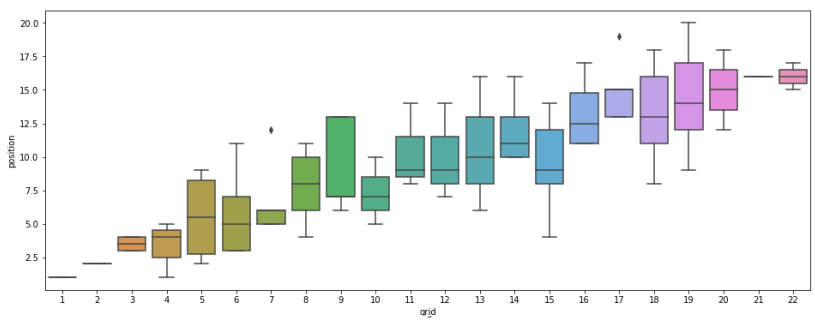

Figure 1: Qualifying position vs Win status

As a F1 car drives at high speeds, it leaves a stream of turbulent air behind it. This turbulent air can heavily decrease the performance of the car behind, as F1 cars are heavily aerodynamic dependent[1]. Having this in mind, a histogram was created to show the observations of winning and losing for each starting position.

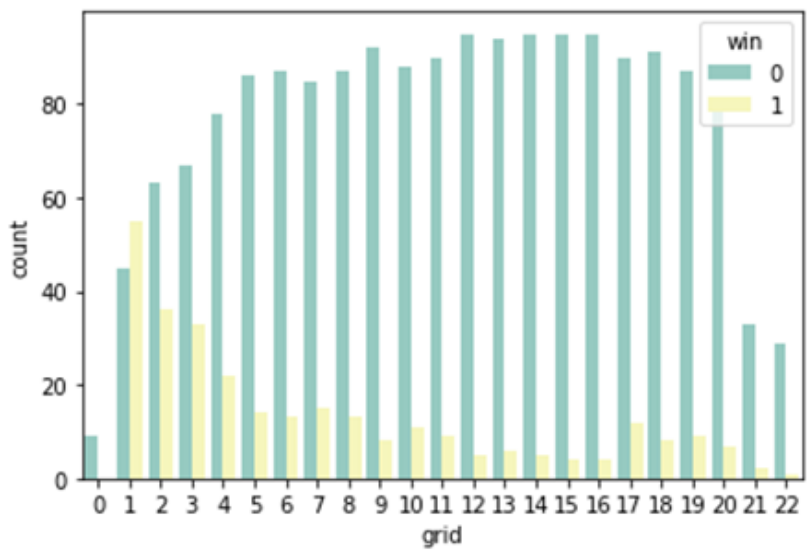

Figure 2: Histogram of win/lose count per starting position

From the histogram, we see that wins are heavily concentrated in the first few starting positions, and it quickly tapers off.

4.2 Age trend of winners

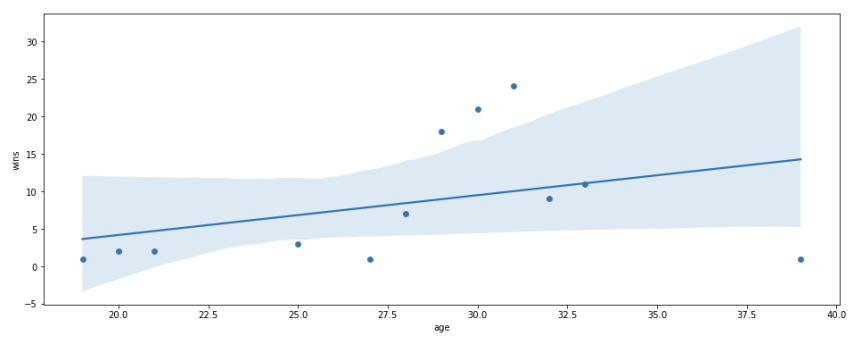

Figure 3: Scatter plot of Win vs Age

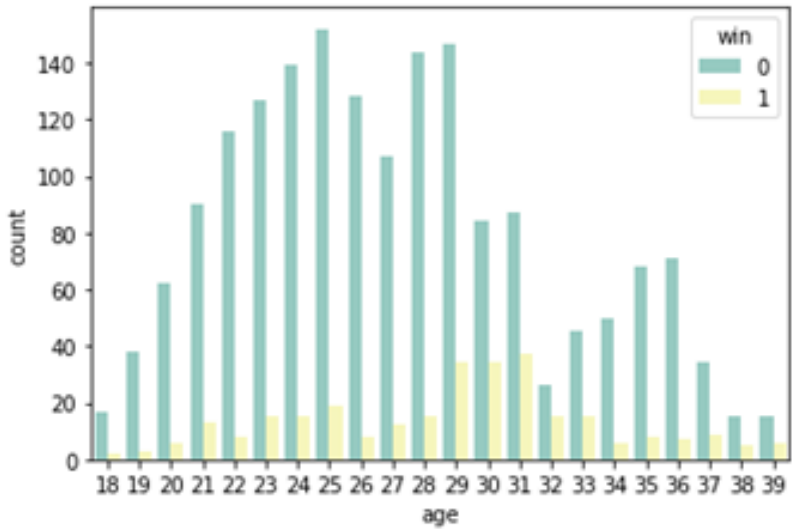

Figure 4: Histogram of win/lose frequency per age

Looking at the histogram, it might suggest that drivers between the age of 28 and 32 might be the ideal age to win races, however this conclusion might be skewed as most of these wins were by one or two drivers who dominated during the interval that we are examining. Furthermore, if we have a look at a simple regression line in the scatter plot, we see that those wins are outside of our confidence interval of 95%.

4.3 Wins vs Teams

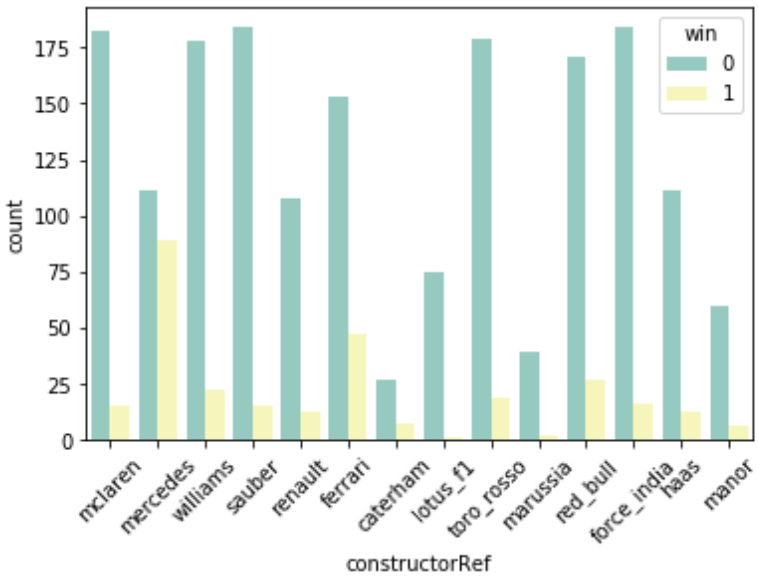

Figure 5: Histogram of win/lose frequency per team

Mercedes by far wins the most races within the interval we are examining.

4.4 Dangerous circuits

While most modern circuits are designed with driver safety and spectator interest in mind, some races are still held on historical street circuits. These circuits generally feature less runoff area, less track width, and also less ideally prepped surface. All of this adds to the danger factor of the track.

For a race team, knowing which track is more dangerous and changing the strategy is very important. When racing on a dangerous track, the driver should put more importance on keeping the car within the track and take less risks while fighting another driver for position. However, because the risk of crashing on a dangerous track is higher, the possibility of a safety car slowing the race down is also higher. If the team times the pitstop correctly, they could drastically improve their overall position in the race. On the flip side, driving on a safer track allows the driver to make more mistakes and fight other drivers for positions more frequently.

To do this, we looked at the number of times any driver is reported to either have an accident or a collision in a particular track.

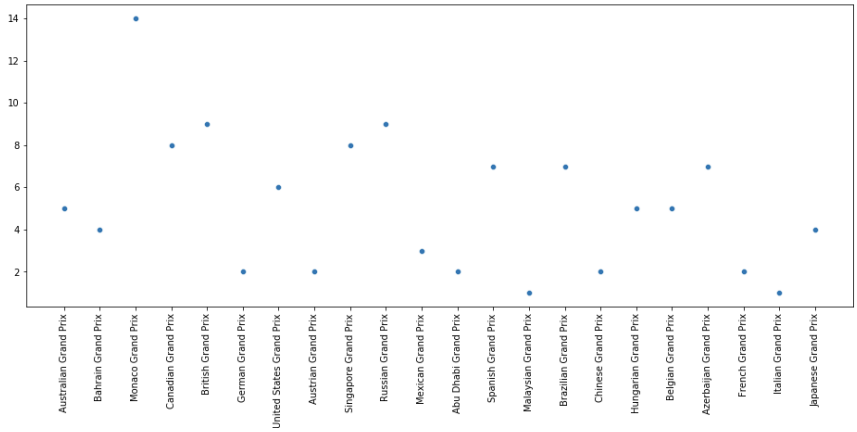

Figure 6: Number of collision or accident at each track

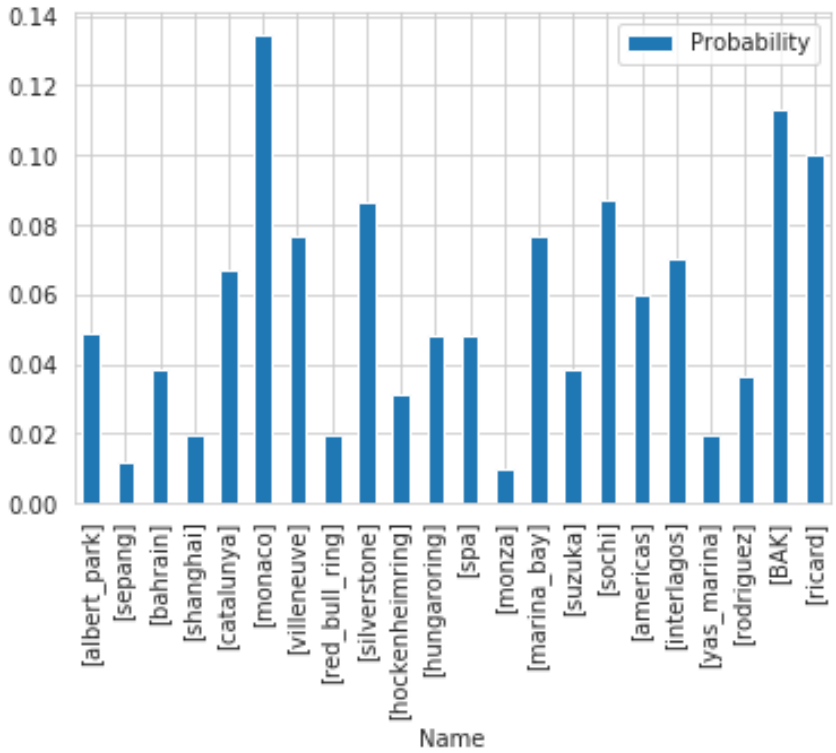

Figure 7: Estimated Probability of collision or accident at each track

Street circuits such as Monaco and Baku top the charts at roughly 13.8% and 11.5% probability of an accident respectively. Tracks designed with overtaking and spectators in mind like Sepang and Shanghai feature much smaller probability at roughly 0.015% and 0.018% respectively. For contrast, the Monaco circuit is only 12 meters at its widest, while Sepang is 16 meters at its narrowest and 20 meters at its widest [2].

5 Regression

5.1 Data preparation

The multiple tables were joined using their primary keys and foreign keys. However, careful preparation had to be done to prevent duplication of entries that would influence the model. After the tables were joined, multiple unused columns such as *url* and *lat/long/alt* were removed.

5.2 Feature engineering

Although the dataset contains tons of data, some of it are not easily understood by computers and require some workarounds to get working. In order to get the best possible regression model, we first create some new columns (features) in the dataset.

The first column we create is *Age*. This column represents the age of the driver during the particular race. The age is calculated by finding the difference between the year of the race and also the year of the drivers date of birth.

The second column we create is *Win*. This column is a binary value representing whether or not the driver achieved pole position for the race. Each race can only have one pole position.

In our final table, we are only left with the following columns: *grid, rank, constructorRef, win, pole, age*, where *grid* is the starting position of the particular race, *rank* is the rank of the fastest lap, *age* is the age of the driver during the race, *constructorRef* is a text representation of the team name, and *pole* is a binary value to represent whether or not the driver achieved pole position during the race.

5.3 Model creation

While it might be tempting to use multiple linear regression to create a model to predict win status, linear regression is unsuitable for this problem as it only produces continuous numerical values. Further, if we have input values that are extremely different than the input values used to train the regression, the output from the numerical model might not make sense contextually.

To overcome this, we chose to use logistic regression instead. Logistic regression models the probability of the observation to belong in a class [3]. The output from the logistic regression are binary values (0=lose, 1=win), which can be used for classification. The domain would be $[-\infty, \infty]$, while the range is $[0, 1]$. Hence, our model would be

$$\ln\left(\frac{Y}{(1-Y)}\right) = b_0 + b_1(grid) + b_2(rank) + b_3(age) + b_4(pole) + b_5(constructorRef) + e$$

Because *constructorRef* is categorical and not numerical, we also had to convert it into dummy variables. This was done using *pandas.get_dummies()* within Python.

We chose the standard 80 20 split for training and testing dataset for this model.

5.4 Model evaluation

5.4.1 Precision and Recall

Precision measures how many selected items by the model are relevant, while Recall measures how many relevant items are selected. They can be interpreted as quality and quantity respectively.

	Precision	recall	f1-score	support
0	0.87	0.99	0.93	1262
1	0.7	0.15	0.24	216
Accuracy			0.87	1478
Macro avg	0.78	0.57	0.59	1478
Weighted avg	0.85	0.87	0.83	1478

Figure 8: Logistic regression model metrics

High precision for class 0 suggests that the model can accurately predict the classification for 0. Further, having a high recall means that most of the relevant items were selected for this category. High precision for class 1 suggests that the model can accurately predict the classification for 1. However the recall is fairly low, suggesting that the model has difficulties selecting relevant items for this category. We suspect that this is due to how we have prepared the data, which resulted in a very imbalanced distribution of the dependent variable.

Notice that the f1-score (f-value) for category 0 is much higher than category 1, suggesting that this model is much better at classifying items that belong to category 0 than it does to category 1.

5.4.2 Confusion Matrix

A confusion matrix is often used to evaluate the performance of a model, as it shows the probability of the model classifying a particular category, but also how accurate it is. From this table, we are able to evaluate the probability of the model predicting false positives, false negatives, true positives and true negatives.

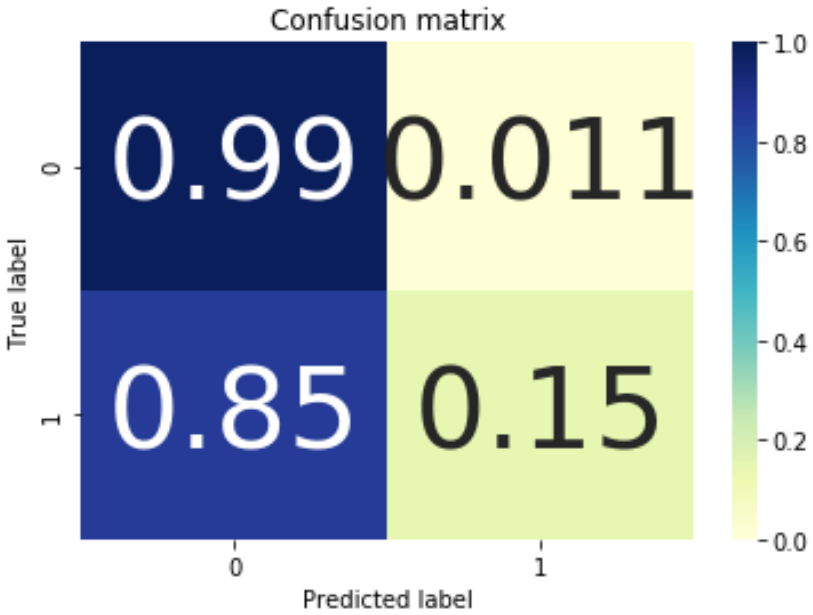

Figure 9: Confusion Matrix of our model

We see that our model is very accurate at predicting true negatives (99%), but struggles at predicting true positives (0.15%) and exhibits telltale signs of a model that produces type-ii errors (85%).

5.4.3 True Positive Rate

True positive rate measures the proportion of those who belong to classification of category 1, and is correctly classified by the model.

	Precision	recall	f1-score	support
0	0.87	0.99	0.93	1262
1	0.7	0.15	0.24	216
Accuracy			0.87	1478
Macro avg	0.78	0.57	0.59	1478
Weighted avg	0.85	0.87	0.83	1478

Figure 10: TPR output

We see that our model has an average TPR of 0.57. This suggests that the model can predict roughly 57% of the items that belong to category 1. This is not a very good value if we wanted to accurately predict the winner.

5.4.4 Log Loss

Log Loss shows how close the predicted probability matches the actual value. The more the predicted probability varies from the actual value, the higher the log loss becomes. Thus, it is in our interest to decrease the log loss of our model.

```
The Log Loss on Training is:  0.3585978816594228
The Log Loss on Testing Dataset is:  0.36995802678434625
```

Figure 11: Log Loss output

We calculated the log loss for both our training dataset and testing dataset. The training dataset log loss shows us the loss for items that the model has already seen, while log loss for the testing dataset is for items that the model has not seen. Log loss for both datasets is fairly low.

5.4.5 Dummy model

A dummy regression model is a classifier function within *scikit-learn* that is used as a baseline for model comparisons. We created a dummy model with the same training data that we used for our logistic regression model.

```
Testing Acc: 0.8787878787878788
Log Loss: 0.3283703278944129
Testing Acc dummy: 0.8637469586374696
Log Loss dummy: 4.7060133287469545
```

Figure 12: Dummy log loss vs Logistic log loss

We see that while the accuracy stays relatively the same, the log loss for our logistic regression model is much much lower than the dummy model.

6 Conclusion

While the model theoretically works, it performs poorly if the goal is to predict a winner. The model is also very likely to produce false negatives, which would defeat the entire purpose of predicting a winner. It classifies most items as 0, which is unsurprising due to the training dataset. Like mentioned previously, the highly skewed data probably contributed to the poor performance of our model. Overall, the model does not perform as desired, unless the goal of the model was to predict if a driver would lose during a particular race.

7 Further developments

There are several avenues for further development into the project. Primarily, more data such as tyre type, weather condition, wind speed, track & ambient temperature and budget should be collected. Each of these could drastically influence the probability of a race win, and also improving our model. In addition, having more data would mean that further feature engineering could be done.

A different classification model and dependent variable could also be used. If we insist on using Logistic Regression, we could instead predict the position of a driver at the end of a race, rather than a binary representation of win or lose. If we insist on only predicting win or lose, other classifications such as K-Nearest Neighbor or decision tree might perform better.

Finally, historical data could be introduced to the model. Historical data could be normalized against current data in a way that would not influence

the training data. This might produce a more accurate model as it has more items to learn from.

8

References

[1] Alto Ono
www.racecar-engineering.com/tech-explained/slipstream-and-dirty-air-explained/

[2] Sepangcircuit.com
www.sepangcircuit.com/facilities/racing-circuits

[3] Thomas W.Edgar, David O.Manz
www.sciencedirect.com/topics/computer-science/logistic-regression

Data sources: *http://ergast.com/mrd/*